

AUTOMATED FAULT DETECTION SYSTEM USING IOT

Submitted in Fulfilment
of the Requirement for the Award
of the Degree Of
BACHELOR OF TECHNOLOGY

In
AEROSPACE ENGINEERING

By
Satendra Yadav - 12111269,
Dev Ashish Sharma - 12116332,
Suyash Thakur – 12110733

Under the guidance
of
Dr. RAHUL KUMAR (20206),
Associate Professor



**DEPARTMENT OF AEROSPACE ENGINEERING,
SCHOOL OF MECHANICAL ENGINEERING,
LOVELY PROFESSIONAL UNIVERSITY,
PHAGWARA, PUNJAB (INDIA) – 144411,**

2024-2025

DECLARATION

We, the undersigned members of the team, hereby declare that the capstone project report titled “**Automated Fault Detection System Using IoT**” is the result of our original work carried out during the academic session 2024–2025, as part of the curriculum for the Bachelor of Technology in Aerospace Engineering at Lovely Professional University, Phagwara.

All information, data, and software used in the project have been duly acknowledged wherever applicable. We affirm that this project has not been submitted earlier, in part or in full, to any other institution or for any other degree or diploma.

We understand that any violation of academic integrity rules may result in disciplinary action.

Team members:

1. Dev Ashish Sharma-12116332
2. Suyash Thakur-12110733
3. Satendra Yadav-12111269

Signatures:

.....
.....
.....



**LOVELY PROFESSIONAL UNIVERSITY
PHAGWARA, PUNJAB - 144411**

CERTIFICATE

I hereby certify that the work which is being presented in the capstone minor entitled “**Automated Fault Detection Using IOT**” in partial fulfillment of the requirement for the award of degree of **Bachelor of Technology** and submitted in Department of Aerospace Engineering, Lovely Professional University, Punjab is an authentic record of my work carried out during period of Capstone Minor under the supervision of **Dr. Rahul Kumar, Associate Professor**, Department of Aerospace Engineering, Lovely Professional University, Punjab.

The matter presented in this capstone minor has not been submitted by us anywhere for the award of any other degree or to any other institute.

Date: 1/05/2025

Satendra Yadav
Suyash Thakur
Dev Ashish Sharma

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date: 1/05/2025

Dr. Rahul Kumar
Associate Professor

Signature of Examiner

ACKNOWLEDGEMENT

We want to express our sincere gratitude to all those who contributed to the successful completion of our project “**Automated Fault Detection Using IOT.**”

We extend our Special thanks to our esteemed supervisor, **Dr.Rahul Kumar**, Associate Professor, Department of Aerospace Engineering, for their invaluable guidance, constant support, and encouragement throughout this project. Their thoughtful ideas and constructive feedback were highly contributive to formulating the path and intensity of our work.

We are also incredibly thankful to Dr. **Amit Kumar Thakur**, Head of the Department of Aerospace Engineering, for giving us the facilities, infrastructure, and academic atmosphere required for the implementation of this project.

ABSTRACT

The report justifies the use of computer programming and software designing to design a website which could ease the work of aircraft MRO companies by acting as a central means of communication between various employees involved in checking, testing ,inspection and repairing of the aircrafts of various types which are registered in the MRO companies. It is not only reliable but there are less chances of error caused by machines as compared to humans. The software is intended to work hand-in-hand with humans to ensure safety and security of passengers overboard. The fault once detected during checking, will be displayed in the panel to be read by the responsible person to be acted upon. Hence it creates a central system by which various workers communicate with each other and assigns each person his responsible. This makes the person more accountable to his work. By simply designing an interface which is user friendly the people can be more aware of any liable faults and malfunctioning. The software is designed in python language and tools play an important part. The software is designed such that even a small vulnerable fault may not go unnoticed.

The python language used to design the software is easy to use and has greater similarity with general human language. Thus there are less chances of error while designing the code.

The whole interface is user friendly with various panes depicting various aspects of condition of each of the aircrafts.

Tools like HTML and Visual Studio Code were used to design the website which were made compatible with python language.

In order to enter details after a particular inspection task is done the website has a facility of “Backend” which is an interface by which the workers can edit the status after they have completed their inspection to be displayed in the main display panel called “Frontend”.

Table of Contents

S.no	CONTENT	Page no
1.	Abstract	5
2.	Introduction	8
3.	Literature review	10
4.	Review and Analysis	14
5.	Methodology	17
6.	Research Gap	18
7.	Objective of the study	19
8.	Code for website development	20
9.	Working Model Snapshots	43
10.	Conclusion	47
11.	Future Work	48
12.	Achievement and Award	49
13.	Reference	50

List of Figures

S.no	Figure Content	Page no
1.	Backend server script handling data storage and communication with the web Dashboard.	43
2.	Maintenance Dashboard	43
3.	Admin panel	44
4.	Maintenance Summary	44
5.	Active Alerts	45
6.	Upcoming Maintenance	45
7.	Fleet Status	46
8.	Scheduled vs Unscheduled	46

1. Introduction

Maintenance is the backbone of engineering lifecycle management, safety, and operational reliability in highly regulated industries such as aerospace, which demand greater performance. Aircraft systems are sophisticated, and failure in a system is highly probable, and therefore the aviation industry demands strict adherence to the maintenance process. Traditionally, technicians documented maintenance inspections, defects, repair, and system changes manually in paper logbooks. Although traceability and compliance were offered in the process in the past, the process has been proven to be deficient in addressing the needs of current aircraft technology and the demand for real-time data. Since they are human input based, they are not empowered with real-time data, and they have no capability to carry out analysis, so manual processes have limited capability, are error-prone, and react slowly[1].

It is the very nature and inherent complexity of the aviation maintenance business, with a wide range of activity outside the traditional A-check, B-check, C-check, and D-check model, that distinguishes it. As each successive higher check is more complex and more thorough, these periodic checks are an integral part of the overall maintenance process. Also included in the overall spectrum of maintenance activities are line maintenance, such as routine overhauls and light work done at the airport gate or hangar between flights, heavy maintenance, such as thorough overhauls, structural inspection, and teardown; unplanned maintenance to repair unexpected faults or defects that emerge in service; and condition-based maintenance, where maintenance can be done only when required by utilizing real-time sensor data to determine the condition of systems and components.

The application of Internet of Things (IoT) technology is a significant innovation in monitoring and controlling maintenance activities in this modern world. This capstone, "Automated Fault Detection Using IoT," aims to replace the traditional manual inspection methods and static documentation processes with a new data-driven approach to fault detection and maintenance monitoring[2]. Through the application of IoT-enabled sensors on critical aircraft components and systems, the project offers real-time monitoring of critical operating parameters such as temperature, pressure, vibration, and electrical status. The sensors feed real-time data into a central processing unit where automated logic or threshold-based algorithms are executed to detect anomalies, performance deviations, or incipient faults.

The value of a data pipeline is greatly added by the inclusion of a well-structured, web-based dashboard, built from the ground up with the newest development environments and tools like Visual Studio Code, HTML, CSS, and JavaScript, with backend support. The dashboard substitutes electronic logbooks and maintenance books with easy-to-understand presentations of the state of the system, history-based analysis-driven insights, automated alerting, and fault classification[3]. Its user interface is technical-user-focused, providing actionable information beneficial for routine maintenance and routine predictive maintenance activities.

Its functioning provides productive advantages. One, it allows technical inspections and tests to be commenced, ranging from line checks before flight to sophisticated structure overhauls, by facts rather than assumptions or scheduled programs. Two, it removes dependence upon human judgment by fault detection and warning, reducing the risk of serious issues being overlooked. Three, it enhances operating efficiency by reducing downtime and allowing the maintenance crew to schedule work from the system's actual state rather than planned estimates.

The solution is one of the latest digital aviation and predictive maintenance technologies, with fleet operators increasingly using sensor analytics, condition monitoring systems, and digital twins to optimize aircraft availability and reduce operating expenses. The project is a vision for the future of aircraft maintenance with the logbook digitalization and connection to real-time IoT data to create a smart, networked, and responsive approach to maintenance[4].

Also, the value of the system is not limited to its application in the aviation sector. The overall concept of IoT-based fault detection is applicable in aircraft maintenance but can be modified to an extremely large number of other industrial processes. The dashboards can be utilized to monitor the performance of CNC machines or robotic arms in the manufacturing sector. The method can be applied in the energy sector to predict overheating in transformers and turbine failure. Axle vibration and brake temperature online monitoring is of the highest importance in the railway sector. The level of cross-industry applicability is one of the reasons that add to the overall value of the installed system, and hence it is an end-to-end system for intelligent maintenance in industry-specific highly mechanical and electromechanical systems.

Another fascinating aspect of the system is that it is capable of utilizing data analytics alongside machine learning algorithms. Although the current system in operation is threshold-based fault detection, the system itself is designed to accommodate more sophisticated diagnostic methods. With sufficient historical data, one can create machine learning algorithms that are capable of picking up on subtle patterns of faults that even human inspectors are unable to spot. An excellent example of this would be a neural network identifying patterns in simultaneous subtle parameter drifts that can cause failure. This would not only make the platform reactive but predictive, thus equipping maintenance staff with data-driven recommendations and risk analysis.

The technology is of high value in the framework of regulatory compliance and sustainability. From a compliance and airworthiness perspective, regulatory bodies like the FAA and EASA are more interested in the use of condition-based maintenance, automated reporting, and electronic record-keeping. With real-time logging and electronic traceability, the dashboard enhances audit efficiency, encourages transparency, and enforces compliance with safety regulations.

Besides, IoT predictive maintenance also promotes environmental sustainability. Sophisticated fault detection reduces wastage of energy, prevents unnecessary utilization of components, and minimizes the environmental cost of unplanned system breakdown, thus facilitating greener operations—an activity in line with the general sustainable technology trend in the aviation industry. The other key characteristic that has been built into the project plan is scalability. The reason the hardware and software components have been made modular is that it becomes feasible to deploy the system on an enormously large number of aircraft, from tiny drones to commercial airliners, without extensive redesign. It is also simple to add more sensors and employ more sophisticated data processing techniques as and when needed. The back-end website architecture has also been designed with future deployments such as cloud storage, remote diagnostics, user-specific access control, and mobile responsiveness in mind. Such forward planning makes the project scalable to an enterprise-level maintenance management system. Briefly, this capstone project illustrates how current digital technologies—IoT and web development, in this case—can be combined to address real-world industrial challenges. It merges mechanical engineering, data science, and user interface design and illustrates how a properly designed digital system can mechanize processes that otherwise continue to rely on human labor and personal judgment. With digitalization of aircraft maintenance procedures and real-time fault detection, the platform significantly enhances operational efficiency, reduces risks, and maximizes system life. As industries increasingly turn toward automation and digital convergence, solutions like these are not just handy—they are a must.

2. Literature Review

The application of intelligent fault detection and maintenance systems has been revolutionized over the last decades. With the emergence of the Internet of Things (IoT), the revolution has picked up unprecedented speed, especially in industries like aerospace, manufacturing, and transport. These industries, based on seamless operation and system reliability, are now adopting IoT-based solutions to detect, diagnose, and act on faults in real-time in growing and growing numbers. This literature review presents an integrative synthesis of seminal academic and industry research on automatic fault detection, IoT integration into maintenance, and web-based dashboard development, with specific focus on applications to aircraft systems.

1. Conventional Methods of Maintenance

Before the emergence of digital technologies, maintenance activities were predominantly of a preventive and corrective nature. Preventive maintenance (PM), even though more formalized, is based on fixed time or usage levels on parts maintenance regardless of their actual condition. Corrective maintenance (CM) is equipment maintenance after it has failed, frequently leading to unplanned downtime and risk.

Most of the PM practices, according to Mobley (2002), were designed to avoid catastrophic failure but resulted in over-maintenance—replacement of parts before they reached their true useful life. While improved upon CM, the practice was time-consuming, expensive, and required a lot of labor. CM and PM failures led to the creation of more flexible methods such as Condition-Based Maintenance (CBM) and Predictive Maintenance (PdM).

2. Condition-Based and Predictive Maintenance

CBM was among the first attempts towards using data in ascertaining the status of assets in real time. There would be sensors to detect temperature, vibration, and pressure, and alarm when levels crossed limits. Jardine et al. (2006) had defined CBM as an asset management revolution that prevented surprise shutdowns by tracking faults in their early stages. CBM was not able to predict failures, and hence its preventive benefit was lost.

Predictive Maintenance (PdM), on the other hand, utilizes experience, machine learning, and sensor data to predict the remaining useful life (RUL) of parts. Lee et al.'s (2014) survey noted that PdM is a significant enhancement from CBM since, apart from fault detection, it also predicts system failures in the future. IoT has significantly facilitated the viability and efficacy of PdM with scalable, real-time streams of data from embedded sensors.

3. IoT's role in modern maintenance systems

IoT has become the technology backbone for contemporary maintenance systems. IoT stands for a collection of intelligent devices that can gather, transfer, and process data in real time. IoT, as Bagheri et al. (2015) define, converts passive machines into intelligent machines that can identify and communicate. With inbuilt intelligent sensors in key systems, data flows in real time to cloud platforms or local servers where it is processed and visualized.

In fault detection, IoT sensors track various operating parameters—engine vibration, oil temperature, hydraulic pressure, electrical current, and airflow rate—and identify patterns that indicate deteriorating conditions. Where data are out of range, automatic alarms can be alerted to the maintenance team. Such IoT-based systems are far more accurate, quicker, and more reliable than manual monitoring, according to Galar et al. (2016).

4. IoT Applications for Aircraft Maintenance

Aircraft maintenance is perhaps the most challenging use of automated fault detection because of the stringent safety requirements,

complex operations, and mission-critical uses. As Sharma and Rathore (2019) suggested, conventional aircraft maintenance was defined by undue dependence on physical logbooks and manual checklists. Despite being thorough, they are time-consuming, error-prone, and scope-limited.

Planes today have tens of sensors that gather flight and ground data. Though not entirely for real-time monitoring, the data are also utilized for maintenance planning. IoT-based aircraft health monitoring products such as Boeing's Airplane Health Management (AHM) and Airbus' Skywise platform are the finest examples. The platforms track engine health, landing gear performance, cabin pressure, etc., and maintenance teams can intervene before failure.

Although these systems are robust, they are proprietary and typically prohibitively expensive for small operators, schools, or start-ups. Researchers such as Bumblauskas et al. (2017) have promoted the development of more low-cost, open-source, and modular systems that can mimic the best features of high-end maintenance systems using low-cost hardware and open-software platforms.

5. Designing Low-Cost IoT Prototypes

Academic researchers have addressed this requirement by developing low-cost, working prototypes of automatic fault detection. Patel and Dey (2019), for example, created a system based on Arduino microcontrollers, vibration sensors, and a Node.js server to track industrial machinery. Their system provided basic real-time alerts and maintenance logs. Ahmed et al. (2020) also employed a Raspberry Pi-based system that took inputs from temperature and current sensors and showed them on a web-based interface.

All these projects make use of open platforms like Firebase, ThingSpeak, and Blynk to store and show data. A research paper by Yan et al. (2021) shows that even such low-level software can provide acceptable response and accuracy for pilot-scale systems. These experiments prove the hypothesis that low-cost and scalable fault detection systems can be built using open-source tools—a design philosophy on which the current capstone project is founded.

6. Significance of Web-Based Dashboards

User interaction and data visualization are just as critical to modern fault detection systems as data acquisition. Dashboards on the web are the primary interface via which users view system

status in real-time, get notifications, view fault histories, and plan maintenance. A good dashboard increases situational awareness and speeds up decision-making in the opinion of Rahman et al. (2021).

The majority of dashboards in current research are built with HTML5, CSS, JavaScript (the React and Vue libraries), and backend programming like Node.js or Python Flask. They also have modules like real-time sensor readings, performance graphs, system health monitors, and fault logbooks. Studies like Mandal et al. (2022) have even integrated dashboards with mobile notifications and SMS notifications so that maintenance staff are notified of faults in real time.

Web-based dashboards can be utilized to fill communications gaps in aviation operations among ground maintenance teams, control centers, and on-board systems. As faults have been detected in mid-air, these can be directly input into the system, and ground work can be initiated for solution prior to the airplane being due—minimizing turnaround time while maximizing fleet availability.

7. Challenges in IoT-Based Maintenance

Though it is laden with benefits, IoT-based fault detection systems also have some problems in their execution. Sensor integrity and data accuracy are some of the most glaring problems. Sensors are exposed to extreme vibrations, temperature changes, and electromagnetic interference in harsh environments such as in aircraft engines or industrial compressors. Faulty or poorly calibrated sensors have a tendency to feed erroneous data, leading to false positives or false negatives.

The second concern being addressed in this case is data privacy and security. Because these systems are propagating sensitive information over networks, they are vulnerable to cyber attacks. According to Kim and Zhao (2019), utmost encryption methods and authentication processes must be used to safeguard data integrity. Standardization and scalability are also of utmost importance. IoT solutions are normally customized and non-interoperable. Zhang et al. (2022) have highlighted the necessity for global standards for sensor interfaces, communications protocols, and data structures to make system integration easier and deploy them in large numbers.

8. Future Directions: Synergy of AI and Machine Learning

Recent research has highlighted the synergy of Machine Learning (ML) and Artificial Intelligence (AI) in fault detection. AI can be used to improve the precision and adaptability of fault prediction models, especially in handling huge amounts of unstructured data. Support vector machines, decision trees, regression analysis, and deep learning are increasingly being used for fault

classification, RUL prediction, and proposing repair tasks. Khandelwal et al. (2023) gave a case study showing how ML-based sensor data fault classifiers were 94% accurate in motor failure prediction against traditional rule-based methods. Such models must then be implemented, however, with computing power and high-quality training data—prerequisites facilitated step by step with the rise of cloud computing and edge processing. Conclusion The literature here captures adequately the vital role IoT is asked to perform in transforming maintenance systems from reactive, manual to smart, proactive ones. From condition monitoring to predictive analytics, sensor technology, real-time dashboards, and cloud infrastructure, technology has revolutionized fault detection across the board. The research community has made significant progress in developing low-cost, open-source systems demonstrating the viability of IoT-based maintenance, especially in academic or resource-poor environments. This capstone project continues these foundation researches through the availability of a tailored, web-based aircraft maintenance dashboard bringing together sensor data, fault alerts, and review of the past. It moves the field of smart maintenance both theoretically and empirically forward, and provides a roadmap to future development, including AI integration, cross-platform compatibility, and multi-user collaboration.

3. Review and Analysis

One of the most significant innovations in modern maintenance management is the shift from traditional to automated fault detection in engineering systems. The necessity for such a shift is driven by the growing complexity of machinery, the need for real-time response, and the need for decision-making based on data. This capstone project, "Automated Fault Detection Using IoT," is an apt response to such growing demands. For our project, we aimed to create and design a smart, web-based system by using IoT sensors for real-time fault detection, specifically for the aviation maintenance sector. The project is more than a technical demonstration of concept; it is a vision of how smart maintenance systems can be made available and implemented in operationally sensitive sectors.

Standard maintenance practices relied on manual checks, paper records, and the skill of the technicians. Aviation logbooks, which were once gold standards for maintenance records, were used extensively to document inspection records, system faults, and the status of various components. The practice, as much as it brought structure and traceability to the letter of the law, had severe disadvantages. Those are the likelihood of human error in data entry, the inability to rapidly access historical records, the delay in propagating growing faults, and the complete lack of predictive capability. Physical logbooks also cannot address the present need for digital connectivity, central databases, or inter-team collaboration. Those are the problems our project is specifically designed to eliminate with smart systems and IoT integration.

One of the most important aspects of this project's analysis is to research current technologies and how we stack up. Our research at the literature review phase pointed out that major industry aviation players such as Boeing and Airbus have implemented sophisticated maintenance

systems—Airbus Skywise and Boeing AHM respectively. These systems are big data analytics, machine learning, and full aircraft digital twin reliant. Although these systems are extremely powerful, they are costly and resource-hungry. They necessitate a complete changeover from legacy infrastructure and massive training. In contrast to our proposed system, our proposed system is scalable and cost-effective. It proves that with modest tools and open-source platforms, it is possible to develop a functional prototype that can monitor fault conditions, visualize real-time data, and act as a predictive maintenance assistant.

Perhaps the most compelling aspect of our system is diagnostic monitoring in real time. The IoT sensors that we used are the ears and eyes of the aircraft system and are monitoring in real time for such parameters as vibration, voltage, pressure, and temperature. These sensors are programmed to raise an alarm if pre-set levels are crossed, so any departure is detected in real time. In contrast to traditional checks—carried out from time to time and sometimes failing to detect faults that develop quickly—our system is continuously on the lookout. This adds to operational safety, but also extends component lifespan by enabling intervention at an early stage.

From the software engineering perspective, the system architecture is strong, flexible, and user-friendly. Designed in HTML, CSS, JavaScript, and integrated via server-side scripting, the dashboard provides a seamless user experience. Designed in Visual Studio Code and device-responsive, the dashboard contains modules for real-time status display, fault monitoring in history, sensor data visualization, and alert notification. Not only functional, they have also been carefully chosen to enhance situational awareness and ease maintenance procedures. In industries like aviation, where turnaround time of aircraft is crucial, a marginal efficiency in fault reporting can save thousands of dollars and save the lives of passengers.

The decision to release the platform as a web-based dashboard and not as a standalone software package was given careful consideration. Web-based dashboards have a number of advantages: cross-platform deployment, centralized maintenance, remote control, and lower installation barriers. Maintenance personnel can access the dashboard on any web-enabled platform, which is useful for multi-site company or in-flight maintenance. The decision is also in the cloud-based platforms and centralized control room for maintenance direction since it is in line with industry trend. Another significant benefit our system provides is its predictive maintenance capability. While this release employs static thresholds for fault detection, the system is capable of being adapted in the future to incorporate machine learning algorithms. Predictive maintenance is more than real-time alerting—it analyzes long-term sensor trends to predict when a component will fail. A gradual increase in vibration amplitude, for instance, might predict incremental wear on a bearing. Future iterations of this system might employ pattern recognition to predict these failures and plan ahead for part replacement, minimizing unplanned downtime and maximizing component life.

Also, our system is very scalable and modular. It can be used on any type of equipment—not just airplanes. It can be used on marine systems, factories, smart cars, or buildings with minimal changes in the code. Same code and generic communication protocols are the reasons for it. It is a very useful feature in real-world applications, where there are typically many systems that coexist with one another and must be monitored with one solution.

From a cost-benefit analysis point of view, our project is very cost-effective in relation to commercial quality maintenance platforms. The software employed was completely open-source and the sensors were chosen for reliability rather than cost. This makes the system very adaptable to small and medium enterprises (SMEs), research labs, or institutions that need to upgrade at minimal cost. The skills learned in this project can also be used as a platform to start further academic research or commercial spin-offs. But no system is perfect. The current system is limited by the number of sensor inputs and isn't cloud-based yet. Plus, the fault logic is rather primitive—threshold crossing. It isn't yet equipped with sophisticated decision-making rules like fuzzy logic, neural nets, or AI-based diagnostic trees. Plus, the alert system can be more sensitive with multi-channel alerts (SMS, email, or mobile push notification). Another direction of future development is security of the dashboard. For actual aviation deployment, data encryption, access control, and network security would be required to assure tamper resistance and protection from unauthorized access. In spite of these limitations, the prototype is a proof-of-concept for low-cost intelligent fault detection. It illustrates how real-time monitoring, digital notification, and centralized reporting may be brought together to enhance system reliability. It illustrates how high-complexity engineering systems may be enhanced through reduced cognitive load through simple user interfaces and quick decision-making. Pedagogically, the project has provided a real-world application of cross-disciplinary learning—combining concepts of electrical engineering (integration of sensors), computer science (programming and user interface), aerospace engineering (understanding of aircraft systems), and systems management (procedure for fault documentation). Such a multi-disciplinary design is critical in introducing engineers to the real-world nature of Industry 4.0. Overall, the "Automated Fault Detection Using IoT" project is more than a technical solution—it is a step towards reimagining the detection of faults, their analysis, and resolution in high-reliability environments. Learning from the project further reinforces the world trend towards digitalization of maintenance processes, reducing reliance on human observation, and moving towards intelligent, self-sustaining systems. With further enhancement, such systems can become the industry norm, marking the end of manual logbooks and the beginning of a new era of intelligent, connected maintenance.

4. Methodology

The methodology for the Automated Fault Detection System utilizing IoT was designed to facilitate real-time monitoring, prompt fault detection, and dependable data management via a

cohesive IoT and web-based platform. This approach encompassed several stages, including requirements analysis, hardware configuration, software creation, sensor integration, data storage, fault detection algorithms, and system evaluation.

The initiative commenced with the determination of critical metrics for assessing the condition of equipment, including temperature, humidity, pressure, and vibration. These metrics were chosen for their capacity to indicate operational irregularities or possible system malfunctions. Sensors like the DHT11, which measures temperature and humidity, along with vibration sensors, were selected for their accessibility, precision, and compatibility with microcontrollers such as the Arduino Uno or ESP32.

During the hardware development stage, sensors were connected to a microcontroller, with the ESP32 being favored due to its integrated Wi-Fi functionality. The microcontroller was programmed via the Arduino IDE to acquire sensor readings and relay the information to a remote server through Wi-Fi. The data was structured and transmitted at consistent intervals utilizing HTTP POST requests, facilitating centralized data aggregation for subsequent analysis.

A backend system was implemented utilizing Node.js, or alternatively Python Flask, to manage data received from IoT devices. This backend server provided RESTful API endpoints that received sensor data, processed it, and stored it in a MySQL database. The database schema was structured to record each reading alongside its timestamp, device ID, and fault status, facilitating efficient retrieval and analysis.

The user interface was developed utilizing HTML and CSS to deliver a clear and informative dashboard. It featured various sections, including System Status, Sensor Logs, Maintenance Summary, and Fault Records. The design prioritized responsiveness and user-friendliness, allowing users to easily access essential data in a clear and efficient manner. The backend system facilitated real-time updates, dynamically refreshing the frontend content through updated API calls.

Fault detection was achieved by establishing threshold limits for each parameter under observation. This logic was integrated into both the microcontroller firmware and the backend system. For example, when the temperature surpassed a predetermined threshold (such as 80°C), the system identified it as a fault and recorded it in the database. Concurrently, the status was displayed on the web dashboard to notify the user. The system was designed to accommodate multiple sensor inputs and consolidate the data for thorough analysis.

In order to assess the performance of the system, simulations of both standard and erroneous scenarios were executed. Systematic tests were performed to confirm that the system effectively identified irregular conditions and recorded faults promptly. The data logs underwent analysis for consistency, and the interface was evaluated to guarantee that all system statuses and alerts were accurately presented.

Following successful testing, the complete system—which encompasses sensors, a microcontroller, a server backend, a database, and a web interface—was implemented on a local network. This facilitated a practical demonstration of real-time fault detection and data monitoring. The architecture is designed to be modular and scalable, enabling future integration with cloud platforms or expansion for industrial and commercial uses.

5. Research Gap

Despite notable progress in fault detection and predictive maintenance through IoT technologies, there remain several research gaps that hinder the efficiency, scalability, and adaptability of these systems. Traditional fault detection approaches predominantly depend on manual inspections or offline data analyses, resulting in delays in fault identification and heightened maintenance expenses. Additionally, current IoT-based systems are frequently tailored for particular applications, lacking the versatility to be applied across various sectors such as industrial machinery, smart homes, or aviation systems.

A significant shortcoming is the insufficient incorporation of real-time monitoring within web-based platforms in affordable and scalable solutions. Numerous current systems either do not provide intuitive dashboards or rely on external cloud services, which can lead to delays, security issues, and elevated operational expenses. Furthermore, many models emphasize data gathering while neglecting to deploy efficient fault detection algorithms at the edge (i.e., directly on the microcontroller), leading to superfluous data transmission and reduced response efficiency.

Furthermore, there is a lack of adequate focus on localized fault detection, which involves identifying and recording faults directly at the sensor level prior to transmission to the server. The lack of this methodology results in inefficiencies within the system, particularly in areas with restricted connectivity. Additionally, IoT fault detection models designed for small-scale or educational purposes seldom include structured data logging and visualization, both of which are essential for comprehensive long-term analysis and informed decision-making.

This initiative aims to create an affordable, scalable IoT-based automated fault detection system. It emphasizes real-time monitoring, localized fault detection with embedded logic, and a custom web dashboard independent of commercial platforms. By combining hardware, backend API connectivity, and an offline-capable web interface, it provides a versatile solution for various scenarios and lays the foundation for future research in intelligent maintenance systems.

6. Objectives Of The Study

The primary aim of this research is to create an Internet of Things (IoT) based Automated Fault Detection System that can continuously monitor both environmental and operational parameters in real-time, identifying any abnormal conditions that could potentially result in system faults or failures. This system is intended to offer an efficient, dependable, and cost-effective solution that lessens the necessity for manual inspections and mitigates the risk of unforeseen breakdowns in industrial, electrical, or mechanical systems. By integrating sensors with a microcontroller and linking the hardware to a web-based platform, this study seeks to facilitate a smooth transfer of data from the physical environment to the digital interface.

The research additionally emphasizes the creation of an adaptive and user-centric web dashboard utilizing HTML and CSS, allowing users to observe real-time sensor data, track system performance, retrieve fault logs, and analyze historical patterns. This interface aims to streamline maintenance procedures and enhance the capacity for data-informed decision-making. Moreover, the goal encompasses the design of a scalable backend framework capable of efficiently processing sensor data and securely storing it in a structured database for subsequent analysis.

The main goal is to enable real-time monitoring and fault detection by setting threshold values for temperature, humidity, and vibration. If these thresholds are exceeded, the system automatically detects and logs the fault, alerts the user, and updates the monitoring dashboard. Another key objective is to incorporate local edge processing, allowing the microcontroller to make quick decisions even with limited network connectivity, enhancing system reliability in remote or offline settings.

The initiative aims to improve modularity and reusability, allowing the system to be easily modified for various applications like industrial machinery, home automation, or energy management. It should adapt to different sensors and operational needs. Ultimately, this research seeks to integrate intelligent monitoring systems into maintenance and reliability engineering, bridging traditional methods with modern IoT automation.

7. Code for Website Development:

1. Frontend Code (ReactJS, Tailwind, CSS):

Node_modules:

1.@ECHO off

2.GOTO start

3.:find_dp0

4.SET dp0=%~dp0

4.EXIT /b

6.:start

7.SETLOCAL

8.CALL :find_dp0

9.IF EXIST "%dp0%\node.exe" (

10.SET "_prog=%dp0%\node.exe"

11.) ELSE (

12. SET "_prog=node"

13.SET PATHEXT=%PATHEXT:;.JS;=;%

14.)

15.

16.endLocal & goto #_undefined_# 2>NUL || title %COMSPEC% &
"%_prog%" "%dp0%\..\browserslist\cli.js" %*

.\bin:

```
1.#!/usr/bin/env pwsh
2.$basedir=Split-Path $MyInvocation.MyCommand.Definition -Parent
3.
4.$exe=""
5.if ($PSVersionTable.PSVersion -lt "6.0" -or $IsWindows) {
6.# Fix case when both the Windows and Linux builds of Node
7.# are installed in the same directory
8.$exe=".exe"
9.}
10.$ret=0
11.if (Test-Path "$basedir/node$exe") {
12.# Support pipeline input
13.if ($MyInvocation.ExpectingInput) {
14.$input | & "$basedir/node$exe" "$basedir/../acorn/bin/acorn" $args
15.} else {
16.& "$basedir/node$exe" "$basedir/../acorn/bin/acorn" $args
17.}
18.$ret=$LASTEXITCODE
19.} else {
20.# Support pipeline input
21.if ($MyInvocation.ExpectingInput) {
22.$input | & "node$exe" "$basedir/../acorn/bin/acorn" $args
24.} else {
25.& "node$exe" "$basedir/../acorn/bin/acorn" $args
26.}
27.$ret=$LASTEXITCODE
```

28. }

29. exit \$ret

.vite\dsp

```
{  
  1. "name": "@ampproject/remapping",  
  2. "version": "2.3.0",  
  3. "description": "Remap sequential sourcemaps through transformations to point  
    at the original source code",  
  4. "keywords": [  
  5. "source",  
  6. "map",  
  7. "remap"  
  8. ],  
  9. "main": "dist/remapping.umd.js",  
 10. "module": "dist/remapping.mjs",  
 11. "types": "dist/types/remapping.d.ts",  
 12. "exports": {  
 13. ".": [  
 14. {  
 15. "types": "./dist/types/remapping.d.ts",  
 16. "browser": "./dist/remapping.umd.js",  
 17. "require": "./dist/remapping.umd.js",  
 18. "import": "./dist/remapping.mjs"  
 19. },  
 20. "./dist/remapping.umd.js"
```

```
21. },
22. "./package.json": "./package.json"
23. },
24. "files": [
25. "dist"
26. ],
27. "author": "Justin Ridgewell <jridgewell@google.com>",
28. "repository": {
29. "type": "git",
30. "url": "git+https://github.com/ampproject/remapping.git"
31. },
32. "license": "Apache-2.0",
33. "engines": {
34. "node": ">=6.0.0"
35. },
36. "scripts": {
37. "build": "run-s -n build:*",
38. "build:rollup": "rollup -c rollup.config.js",
39. "build:ts": "tsc --project tsconfig.build.json",
40. "lint": "run-s -n lint:*",
41. "lint:prettier": "npm run test:lint:prettier -- --write",
42. "lint:ts": "npm run test:lint:ts -- --fix",
43. "prebuild": "rm -rf dist",
44. "prepublishOnly": "npm run preversion",
45. "preversion": "run-s test build",
46. "test": "run-s -n test:lint test:only",
47. "test:debug": "node --inspect-brk node_modules/.bin/jest --runInBand",
```

```
48. "test:lint": "run-s -n test:lint:*",
49. "test:lint:prettier": "prettier --check '{src,test}/**/*.ts'",
50. "test:lint:ts": "eslint '{src,test}/**/*.ts'",
51. "test:only": "jest --coverage",
52. "test:watch": "jest --coverage --watch"
53. },
54. "devDependencies": {
55.   "@rollup/plugin-typescript": "8.3.2",
56.   "@types/jest": "27.4.1",
57.   "@typescript-eslint/eslint-plugin": "5.20.0",
58.   "@typescript-eslint/parser": "5.20.0",
59.   "eslint": "8.14.0",
60.   "eslint-config-prettier": "8.5.0",
61.   "jest": "27.5.1",
62.   "jest-config": "27.5.1",
63.   "npm-run-all": "4.1.5",
64.   "prettier": "2.6.2",
65.   "rollup": "2.70.2",
66.   "ts-jest": "27.1.4",
67.   "tslib": "2.4.0",
68.   "typescript": "4.6.3"
69. },
70. "dependencies": {
71.   "@jridgewell/gen-mapping": "^0.3.5",
72.   "@jridgewell/trace-mapping": "^0.3.24"
73. }
74. }
```


2. Admin Panel (HTML, ReactJS, Tailwind, Node.js):

.\Public

1. import {
2. import {
3. import {
4. require_react
5. } from "./chunk-YJYY6GXC.js";
6. import {
7. __commonJS
8. } from "./chunk-G3PMV62Z.js";
9. // node_modules/react-dom/cjs/react-dom.development.js
10. var require_react_dom_development = __commonJS({
11. "node_modules/react-dom/cjs/react-dom.development.js"(exports) {
12. "use strict";
13. (function() {
14. function noop() {

```

15. }

16. function testStringCoercion(value) {

17. return "" + value;

18. }

19. function createPortal$1(children, containerInfo, implementation) {

20. var key = 3 < arguments.length && void 0 !== arguments[3] ? arguments[3] : null;

21. try {

22. testStringCoercion(key);

23. var JSCompiler_inline_result = false;

24. } catch (e) {

25. JSCompiler_inline_result = true;

26. }

27. JSCompiler_inline_result && (console.error(

28. "The provided key is an unsupported type %s. This value must be coerced to a string
    before using it here.",

29. "function" === typeof Symbol && Symbol.toStringTag && key[Symbol.toStringTag]
    || key.constructor.name || "Object"

30. ), testStringCoercion(key));

31. return {

32. $$typeof: REACT_PORTAL_TYPE,

33. key: null == key ? null : "" + key,

34. children,

35. containerInfo,

36. implementation

37. };

38. }

39. function getCrossOriginStringAs(as, input) {

40. if ("font" === as) return "";

```

```

41. if ("string" === typeof input)
42. return "use-credentials" === input ? input : "";
43. }
44. function getValueDescriptorExpectingObjectForWarning(thing) {
45. return null === thing ? "`null`" : void 0 === thing ? "`undefined`" : "" === thing ? "an
    empty string" : 'something with type ' + typeof thing + '';
46. }
47. function getValueDescriptorExpectingEnumForWarning(thing) {
48. return null === thing ? "`null`" : void 0 === thing ? "`undefined`" : "" === thing ? "an
    empty string" : "string" === typeof thing ? JSON.stringify(thing) : "number" ===
    typeof thing ? "" + thing + "`" : 'something with type ' + typeof thing + '';
49. }
50. function resolveDispatcher() {
51. var dispatcher = ReactSharedInternals.H;
52. null === dispatcher && console.error(
53. "Invalid hook call. Hooks can only be called inside of the body of a function component.
    This could happen for one of the following reasons:\n1. You might have mismatching
    versions of React and the renderer (such as React DOM)\n2. You might be breaking
    the Rules of Hooks\n3. You might have more than one copy of React in the same
    app\nSee https://react.dev/link/invalid-hook-call for tips about how to debug and fix
    this problem."
54. );
55. return dispatcher;
56. }
57. "undefined" !== typeof __REACT_DEVTOOLS_GLOBAL_HOOK__ && "function"
    ===
    __REACT_DEVTOOLS_GLOBAL_HOOK__.registerInternalModuleStart
    &&
    __REACT_DEVTOOLS_GLOBAL_HOOK__.registerInternalModuleStart(Error());
58. var React = require_react(), Internals = {
59. d: {
60. f: noop,

```

```

61. r: function() {
62. throw Error(
63. "Invalid form element. requestFormReset must be passed a form that was rendered by
    React."
64. );
65. },
66. D: noop,
67. C: noop,
68. L: noop,
69. m: noop,
70. X: noop,
71. S: noop,
72. M: noop
73. },
74. p: 0,
75. findDOMNode: null
76. }, REACT_PORTAL_TYPE = Symbol.for("react.portal"), ReactSharedInternals =
    React.__CLIENT_INTERNALS_DO_NOT_USE_OR_WARN_USERS_THEY_CANNOT_UPGRADE;
77. "function" === typeof Map && null !== Map.prototype && "function" === typeof
    Map.prototype.forEach && "function" === typeof Set && null !== Set.prototype &&
    "function" === typeof Set.prototype.clear && "function" === typeof
    Set.prototype.forEach || console.error(
78. "React depends on Map and Set built-in types. Make sure that you load a polyfill in
    older browsers. https://reactjs.org/link/react-polyfills"
79. );
80. exports.__DOM_INTERNALS_DO_NOT_USE_OR_WARN_USERS_THEY_CANNOT_UPGRADE = Internals;
81. exports.createPortal = function(children, container) {
82. var key = 2 < arguments.length && void 0 !== arguments[2] ? arguments[2] : null;

```

```

83. if (!container || 1 !== container.nodeType && 9 !== container.nodeType && 11 !==
    container.nodeType)

84. throw Error("Target container is not a DOM element.");

85. return createPortal$1(children, container, null, key);

86. };

87. exports.flushSync = function(fn) {

88. var  previousTransition  =  ReactSharedInternals.T,  previousUpdatePriority  =
    Internals.p;

89. try {

90. if (ReactSharedInternals.T = null, Internals.p = 2, fn)

91. return fn();

92. } finally {

93. ReactSharedInternals.T = previousTransition, Internals.p = previousUpdatePriority,
    Internals.d.f() && console.error(

94. "flushSync was called from inside a lifecycle method. React cannot flush when React
    is already rendering. Consider moving this call to a scheduler task or micro task."

95. );

96. }

97. };

98. exports.preconnect = function(href, options) {

99. "string" === typeof href && href ? null != options && "object" !== typeof options ?
    console.error(

100.     "ReactDOM.preconnect(): Expected the `options` argument (second) to be an
    object but encountered %s instead. The only supported option at this time is
    `crossOrigin` which accepts a string.",

101.     getValueDescriptorExpectingEnumForWarning(options)

102.     ) : null != options && "string" !== typeof options.crossOrigin &&
    console.error(

103.     "ReactDOM.preconnect(): Expected the `crossOrigin` option (second
    argument) to be a string but encountered %s instead. Try removing this option or
    passing a string value instead.",

```

```

104.     getValueDescriptorExpectingObjectForWarning(options.crossOrigin)
105.     ) : console.error(
106.         "ReactDOM.preconnect(): Expected the `href` argument (first) to be a non-
            empty string but encountered %s instead.",
107.         getValueDescriptorExpectingObjectForWarning(href)
108.     );
109.     "string" === typeof href && (options ? (options = options.crossOrigin, options
        = "string" === typeof options ? "use-credentials" === options ? options : "" : void 0) :
        options = null, Internals.d.C(href, options));
110.     };
111.     exports.prefetchDNS = function(href) {
112.         if ("string" !== typeof href || !href)
113.             console.error(
114.                 "ReactDOM.prefetchDNS(): Expected the `href` argument (first) to be a non-
                    empty string but encountered %s instead.",
115.                 getValueDescriptorExpectingObjectForWarning(href)
116.             );
117.         else if (1 < arguments.length) {
118.             var options = arguments[1];
119.             "object" === typeof options && options.hasOwnProperty("crossOrigin") ?
                console.error(
120.                    "ReactDOM.prefetchDNS(): Expected only one argument, `href`, but
                        encountered %s as a second argument instead. This argument is reserved for future
                        options and is currently disallowed. It looks like the you are attempting to set a
                        crossOrigin property for this DNS lookup hint. Browsers do not perform DNS queries
                        using CORS and setting this attribute on the resource hint has no effect. Try calling
                        ReactDOM.prefetchDNS() with just a single string argument, `href`.",
121.                    getValueDescriptorExpectingEnumForWarning(options)
122.                ) : console.error(
123.                    "ReactDOM.prefetchDNS(): Expected only one argument, `href`, but
                        encountered %s as a second argument instead. This argument is reserved for future

```

options and is currently disallowed. Try calling ReactDOM.prefetchDNS() with just a single string argument, `href`.",

```
124.     getValueDescriptorExpectingEnumForWarning(options)
125.   );
126.   }
127.   "string" === typeof href && Internals.d.D(href);
128.   };
129.   exports.preinit = function(href, options) {
130.     "string" === typeof href && href ? null == options || "object" !== typeof options
      ? console.error(
131.       "ReactDOM.preinit(): Expected the `options` argument (second) to be an object
        with an `as` property describing the type of resource to be preinitialized but encountered
        %s instead.",
132.       getValueDescriptorExpectingEnumForWarning(options)
133.     ) : "style" !== options.as && "script" !== options.as && console.error(
134.       'ReactDOM.preinit(): Expected the `as` property in the `options` argument
        (second) to contain a valid value describing the type of resource to be preinitialized but
        encountered %s instead. Valid values for `as` are "style" and "script".',
135.       getValueDescriptorExpectingEnumForWarning(options.as)
136.     ) : console.error(
137.       "ReactDOM.preinit(): Expected the `href` argument (first) to be a non-empty
        string but encountered %s instead.",
138.       getValueDescriptorExpectingObjectForWarning(href)
139.     );
140.     if ("string" === typeof href && options && "string" === typeof options.as) {
141.       var as = options.as, crossOrigin = getCrossOriginStringAs(as,
        options.crossOrigin), integrity = "string" === typeof options.integrity ?
        options.integrity : void 0, fetchPriority = "string" === typeof options.fetchPriority ?
        options.fetchPriority : void 0;
142.       "style" === as ? Internals.d.S(
143.         href,
```

```

144.     "string" === typeof options.precedence ? options.precedence : void 0,
145.     {
146.       crossOrigin,
147.       integrity,
148.       fetchPriority
149.     }
150.   ): "script" === as && Internals.d.X(href, {
151.     crossOrigin,
152.     integrity,
153.     fetchPriority,
154.     nonce: "string" === typeof options.nonce ? options.nonce : void 0
155.   });
156.   }
157.   };
158.   exports.preinitModule = function(href, options) {
159.     var encountered = "";
160.     "string" === typeof href && href || (encountered += " The `href` argument
      encountered was " + getValueDescriptorExpectingObjectForWarning(href) + ".");
161.     void 0 !== options && "object" !== typeof options ? encountered += " The
      `options` argument encountered was " +
      getValueDescriptorExpectingObjectForWarning(options) + " : options && "as" in
      options && "script" !== options.as && (encountered += " The `as` option encountered
      was " + getValueDescriptorExpectingEnumForWarning(options.as) + ".");
162.     if (encountered)
163.       console.error(
164.         "ReactDOM.preinitModule(): Expected up to two arguments, a non-empty
          `href` string and, optionally, an `options` object with a valid `as` property.%s",
165.         encountered
166.       );

```



```

167.     else
168.         switch (encountered = options && "string" === typeof options.as ? options.as :
            "script", encountered) {
169.             case "script":
170.                 break;
171.             default:
172.                 encountered = getValueDescriptorExpectingEnumForWarning(encountered),
                    console.error(
173.                     'ReactDOM.preinitModule(): Currently the only supported "as" type for this
                        function is "script" but received "%s" instead. This warning was generated for `href`
                        "%s". In the future other module types will be supported, aligning with the import-
                        attributes proposal. Learn more here: (https://github.com/tc39/proposal-import-
                        attributes)',
174.                     encountered,
175.                     href
176.                 );
177.             }
178.             if ("string" === typeof href)
179.                 if ("object" === typeof options && null !== options) {
180.                     if (null == options.as || "script" === options.as)
181.                         encountered = getCrossOriginStringAs(
182.                             options.as,
183.                             options.crossOrigin
184.                         ), Internals.d.M(href, {
185.                             crossOrigin: encountered,
186.                             integrity: "string" === typeof options.integrity ? options.integrity : void 0,
187.                             nonce: "string" === typeof options.nonce ? options.nonce : void 0
188.                         });
189.                 } else null == options && Internals.d.M(href);

```

```

190.     };
191.     exports.preload = function(href, options) {
192.         var encountered = "";
193.         "string" === typeof href && href || (encountered += " The `href` argument
            encountered was " + getValueDescriptorExpectingObjectForWarning(href) + ".");
194.         null == options || "object" !== typeof options ? encountered += " The `options`
            argument encountered was " +
            getValueDescriptorExpectingObjectForWarning(options) + "." : "string" === typeof
            options.as && options.as || (encountered += " The `as` option encountered was " +
            getValueDescriptorExpectingObjectForWarning(options.as) + ".");
195.         encountered && console.error(
196.             'ReactDOM.preload(): Expected two arguments, a non-empty `href` string and
            an `options` object with an `as` property valid for a `

```

```

211.     referrerPolicy: "string" === typeof options.referrerPolicy ?
        options.referrerPolicy : void 0,
212.     imageSrcSet: "string" === typeof options.imageSrcSet ? options.imageSrcSet :
        void 0,
213.     imageSizes: "string" === typeof options.imageSizes ? options.imageSizes :
        void 0,
214.     media: "string" === typeof options.media ? options.media : void 0
215.   });
216.   }
217.   };
218.   exports.preloadModule = function(href, options) {
219.     var encountered = "";
220.     "string" === typeof href && href || (encountered += " The `href` argument
        encountered was " + getValueDescriptorExpectingObjectForWarning(href) + ".");
221.     void 0 !== options && "object" !== typeof options ? encountered += " The
        `options` argument encountered was " +
        getValueDescriptorExpectingObjectForWarning(options) + " : options && "as" in
        options && "string" !== typeof options.as && (encountered += " The `as` option
        encountered was " + getValueDescriptorExpectingObjectForWarning(options.as) +
        ".");
222.     encountered && console.error(
223.       'ReactDOM.preloadModule(): Expected two arguments, a non-empty `href`
        string and, optionally, an `options` object with an `as` property valid for a `

```

```

230.     as: "string" === typeof options.as && "script" !== options.as ? options.as : void
    0,
231.     crossOrigin: encountered,
232.     integrity: "string" === typeof options.integrity ? options.integrity : void 0
233.   ))) : Internals.d.m(href));
234.   };
235.   exports.requestFormReset = function(form) {
236.     Internals.d.r(form);
237.   };
238.   exports.unstable_batchedUpdates = function(fn, a) {
239.     return fn(a);
240.   };
241.   exports.useFormState = function(action, initialState, permalink) {
242.     return resolveDispatcher().useFormState(action, initialState, permalink);
243.   };
244.   exports.useFormStatus = function() {
245.     return resolveDispatcher().useHostTransitionStatus();
246.   };
247.   exports.version = "19.0.0";
248.   "undefined" !== typeof __REACT_DEVTOOLS_GLOBAL_HOOK__ &&
    "function" ===                                     typeof
    __REACT_DEVTOOLS_GLOBAL_HOOK__.registerInternalModuleStop &&
    __REACT_DEVTOOLS_GLOBAL_HOOK__.registerInternalModuleStop(Error());
249.   })());
250.   }
251.   });
252.   // node_modules/react-dom/index.js

```

```

253.     var require_react_dom = __commonJS({
254.       "node_modules/react-dom/index.js"(exports, module) {
255.         if (false) {
256.           checkDCE();
257.           module.exports = null;
258.         } else {
259.           module.exports = require_react_dom_development();
260.         }
261.       }
262.     });

263.     export {
264.       require_react_dom
265.     };
266.     /*! Bundled license information:

267.     react-dom/cjs/react-dom.development.js:
268.     (**
269.     • @license React
270.     • react-dom.development.js
271.     *
272.     • Copyright (c) Meta Platforms, Inc. and affiliates.
273.     *
274.     • This source code is licensed under the MIT license found in the
275.     • LICENSE file in the root directory of this source tree.
276.     *)
277.     */

```

```
//# sourceMappingURL=chunk-3M4ZFO5U.js.map
```

2. Backend Code (Node.js, Express.js):

.\Alerts.js

```
1. import {
2. import {

3. import { Alert } from '../models.js'
4. import express from 'express';

5. const router = express.Router();
6. // Get all alerts
7. router.get('/', async (req, res) => {
8. try {
9. const alerts = await Alert.find().sort({ severity: -1 });
10.   res.json(alerts || []);
11. } catch (err) {
12.   res.status(500).json({ message: err.message });
13. }
14. });

15. // Create alert
16. router.post('/', async (req, res) => {
17. try {
18.   const alert = new Alert(req.body);
19.   const newAlert = await alert.save();
```

```
20.   res.status(201).json(newAlert);
21.   } catch (err) {
22.     res.status(400).json({ message: err.message });
23.   }
24.   });

25.   // Update alert
26.   router.put('/:id', async (req, res) => {
27.     try {
28.       const alert = await Alert.findByIdAndUpdate(
29.         req.params.id,
30.         req.body,
31.         { new: true }
32.       );
33.       res.json(alert);
34.     } catch (err) {
35.       res.status(400).json({ message: err.message });
36.     }
37.   });

38.   // Delete alert
39.   router.delete('/:id', async (req, res) => {
40.     try {
41.       await Alert.findByIdAndDelete(req.params.id);
42.       res.json({ message: 'Alert deleted' });
43.     } catch (err) {
44.       res.status(500).json({ message: err.message });
```

```
45.   }
46.   });

47.   export default router
```

.\fleet.js

```
1. import {
2. import { Fleet } from "../models.js"
3. import express from "express";

4. const router = express.Router();
5. // Get all fleet status
6. router.get("/", async (req, res) => {
7. try {
8. const fleet = await Fleet.find();
9. res.json(fleet || []);
10.   } catch (err) {
11.     res.status(500).json({ message: err.message });
12.   }
13. });

14. // Create fleet status
15. router.post("/", async (req, res) => {
16. try {
17. const fleet = new Fleet(req.body);
18. const newFleet = await fleet.save();
19. res.status(201).json(newFleet);
```



```
20.   } catch (err) {
21.     res.status(400).json({ message: err.message });
22.   }
23. });

24.   // Update fleet status
25.   router.put("/:id", async (req, res) => {
26.     try {
27.       const fleet = await Fleet.findByIdAndUpdate(req.params.id, req.body, {
28.         new: true,
29.       });
30.       res.json(fleet);
31.     } catch (err) {
32.       res.status(400).json({ message: err.message });
33.     }
34.   });

35.   // Delete fleet status
36.   router.delete("/:id", async (req, res) => {
37.     try {
38.       await Fleet.findByIdAndDelete(req.params.id);
39.       res.json({ message: "Fleet status deleted" });
40.     } catch (err) {
41.       res.status(500).json({ message: err.message });
42.     }
43.   });
```

44. export default router

8. Working Model Snapshot:

```

Backend routes > // fleet.js
1 import { fleet } from './models.js'
2 import express from 'express'
3
4 const router = express.Router()
5 // get all fleet status
6 router.get('/', async (req, res) => {
7   try {
8     const fleet = await fleet.find()
9     res.json(fleet)
10  } catch (err) {
11    res.status(500).json({ message: err.message })
12  }
13 })
14
15 // create fleet status
16 router.post('/', async (req, res) => {
17   try {
18     const fleet = new fleet(req.body)
19     const newfleet = await fleet.save()
20     res.status(201).json(newfleet)
21   } catch (err) {
22     res.status(500).json({ message: err.message })
23   }
24 })
25
26 export default router

```

Terminal Output:

```

PS C:\Users\bell\Downloads\hologram> npm run dev
node_modules\mongoose\lib\mongoose.js:12
  const mongoose = exports = module.exports = {}
  ^
Error: Cannot find module 'mongoose'
    at Module._resolveFilename (node:internal/modules/cjs/loader:1028:15)
    at Module._load (node:internal/modules/cjs/loader:873:27)
    at Module.require (node:internal/modules/cjs/loader:1047:17)
    at require (node:internal/modules/cjs/helpers:102:18)
    at Object.<anonymous> (C:\Users\bell\Downloads\hologram\backend\fleet.js:1:1)
    at Module._compile (node:internal/modules/cjs/loader:1063:14)
    at Module._extensions..js (node:internal/modules/cjs/loader:1081:16)
    at Module.load (node:internal/modules/cjs/loader:915:26)
    at Function.Module._load (node:internal/modules/cjs/loader:728:24)
    at Function.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:81:12)
    at node:internal/main/run_main_module:22:47

```

Figure 1 Backend server script handling data storage and communication with the web Dashboard.

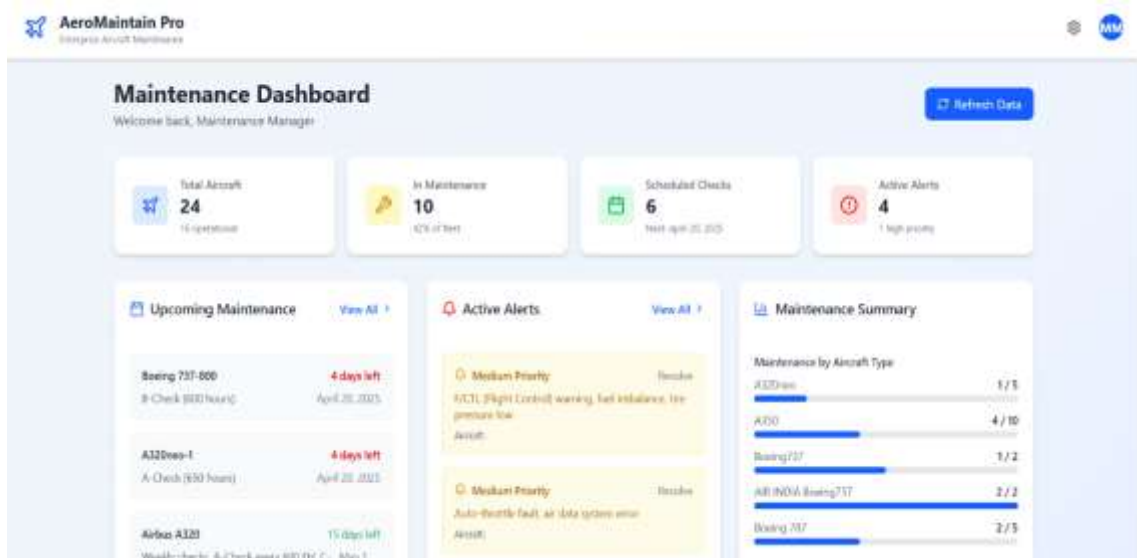


Figure 2 Maintenance Dashboard

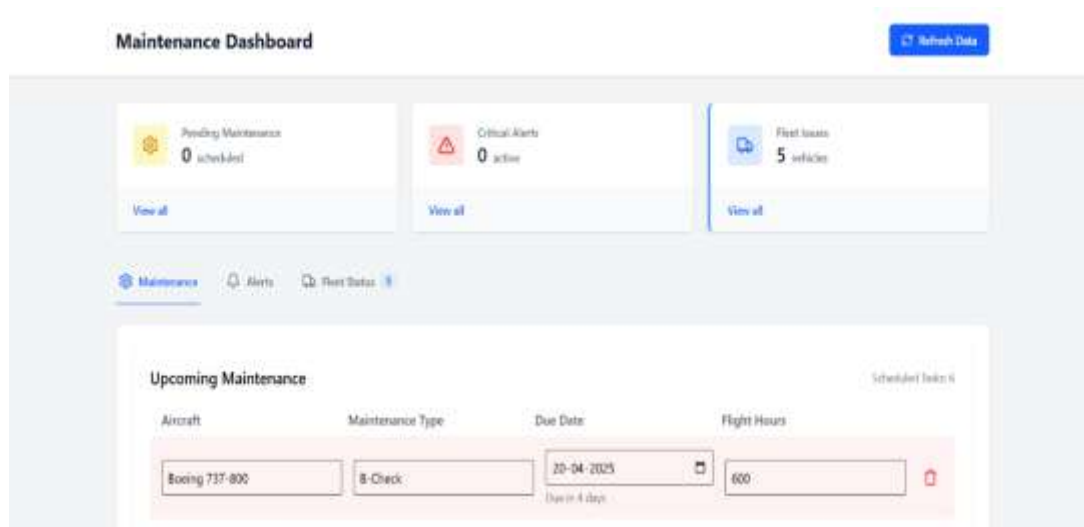


Figure 3 Admin Panel

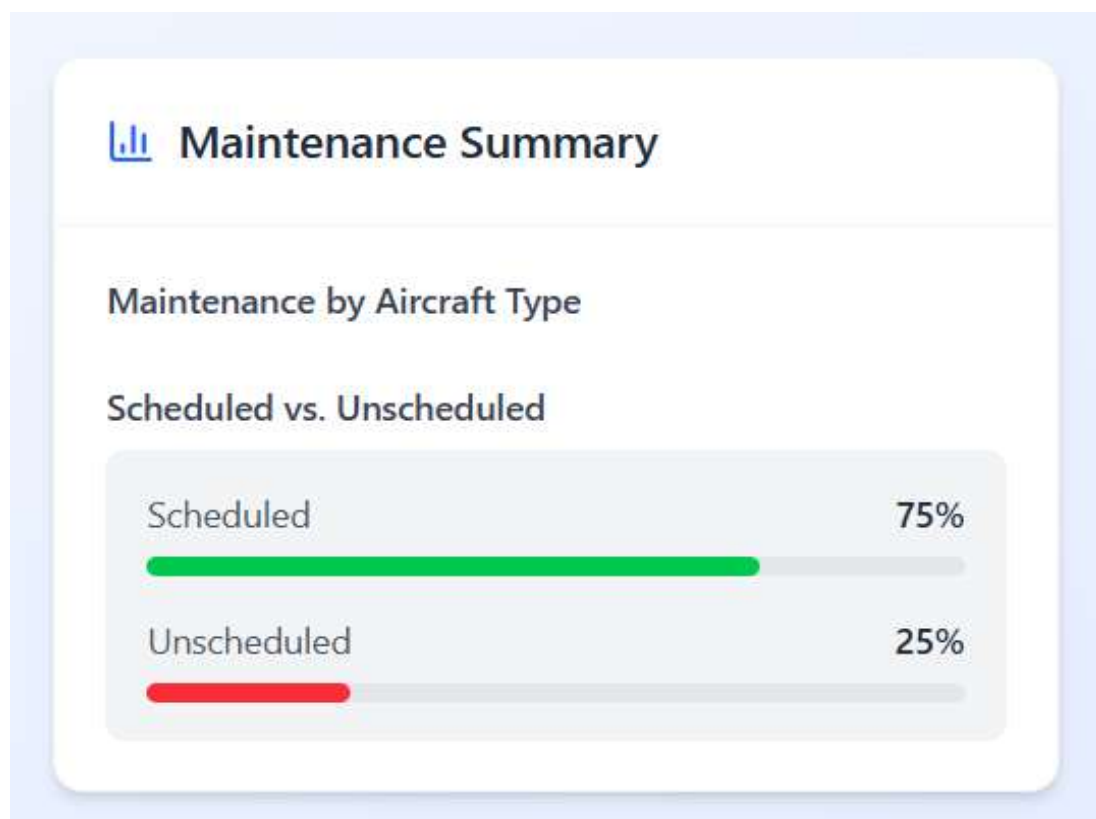


Figure 4 Maintenance Summary

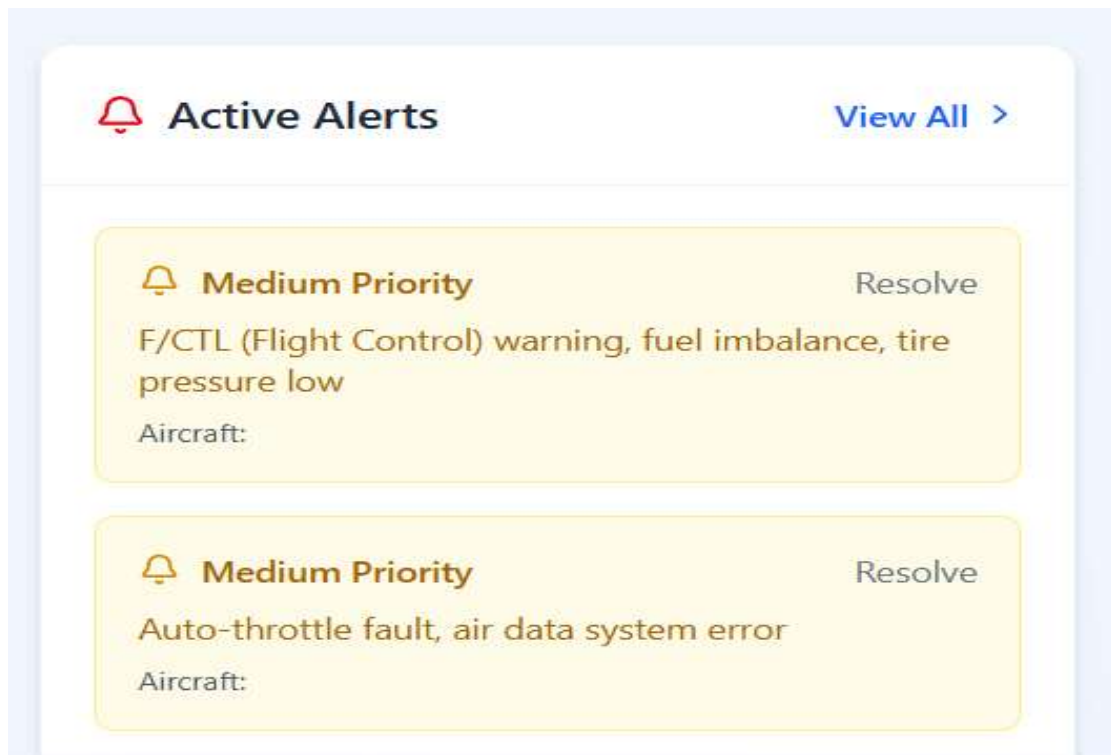


Figure 5 Active Alerts



Figure 6 Upcoming Maintenance

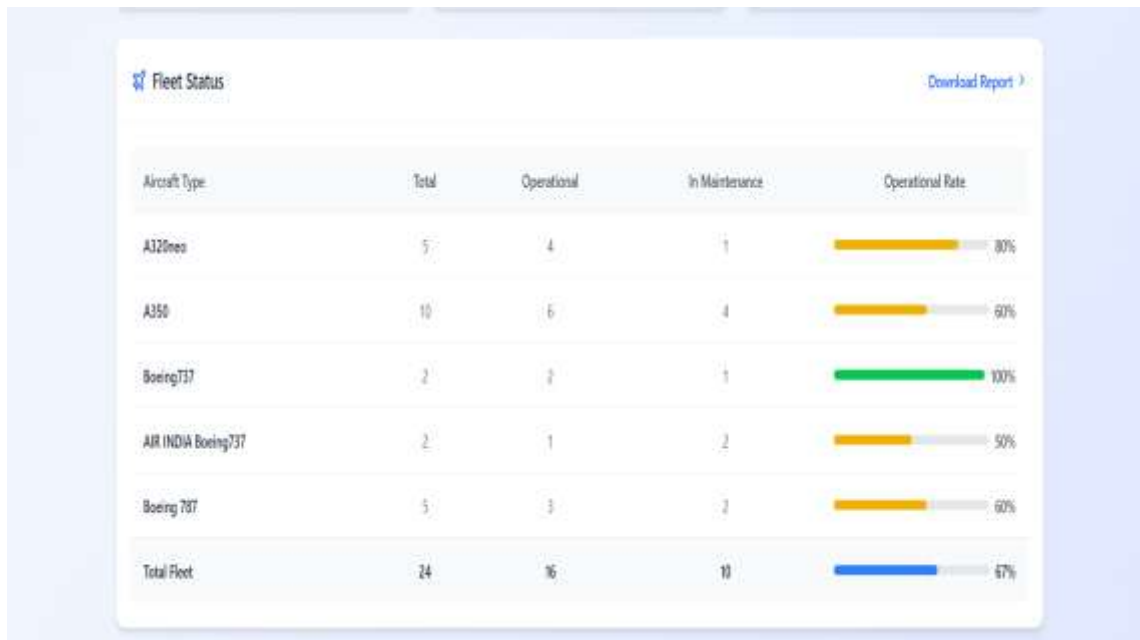


Figure 7 Fleet Status

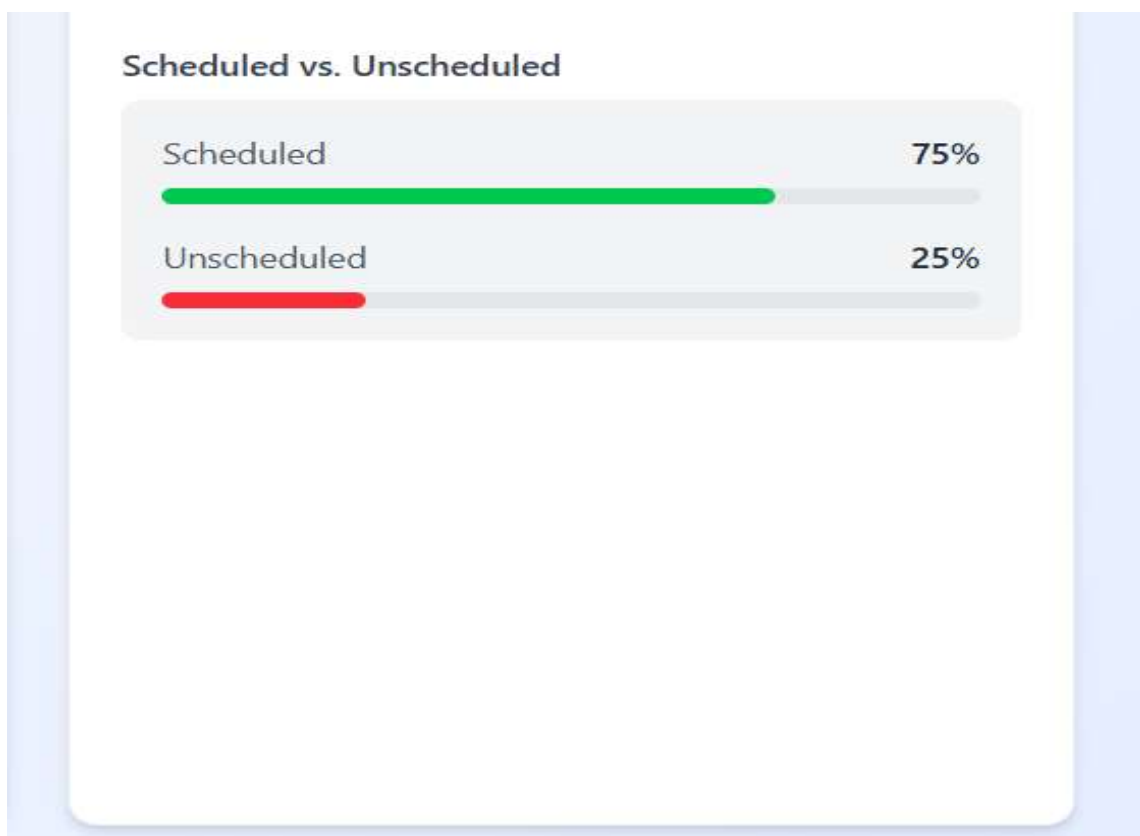


Figure 8 Scheduled vs Unscheduled

9. Conclusion:

The advancements in the last two decades in the technology sector have transformed the engineering systems' frontiers, more specifically the area of fault-detection and maintenance. As operation requirements are going up and as industries with higher reliability, like aerospace, grow in sophistication, the conventional approach of maintenance in the past in terms of periodic inspection and earlier logbook-type approaches cannot compare to the expectations of safety, efficiency, and responsiveness. IOT-based Automated Fault Detection The IoT capstone project has proved to be an effective tool in order to respond to such evolving requirements, marrying a smart, cost-effective, scalable solution with real-time data collection and thoughtful analysis in terms of a web-based expertise platform.

With the assistance of the Internet of Things (IoT), the project demonstrates with tangible evidence how sensor networks can be utilized to bring maintenance systems together in a way that provides real-time insight into equipment condition. The ability of the system to detect anomalies in real time—by leveraging streamed real-time operating data—is a strong departure from the interval inspection paradigms that have long prevailed. In addition, the dashboard we implemented not only serves as a centralized command center but also provides timely notifications and actionable analytics to the maintenance staff, reducing the opportunity for unplanned downtime and halting the advancement of minor faults to total failure. During the design and development phase, the project was high-priority with usability, scalability, and flexibility. With fully open-source tools and coded with Visual Studio Code, the dashboard has a clean, intuitive user interface that is easy to use with multiple levels of technical proficiency. As a web app, its web-based nature offers location and device accessibility to allow the maintenance staff to communicate and make decisions easily. Although our current system is designed for aircraft maintenance, the modularity of our system makes it extremely flexible to implement in other industries like manufacturing, automotive, naval, and infrastructure monitoring. Although the current system lacks prediction modules and end-level AI, it has a good base on which it can be developed to incorporate upgrades. The system can be enhanced with machine learning methods for predictive maintenance, multi-sensor correlation for improved fault classification, and end-level security features for real-world application. All these would not only enhance the capacity of the system but also take it to industrial-grade level fault detection systems employed by large-scale aerospace firms.

Overall, this project is a stepping stone to the future of maintenance engineering—a future where data, automation, and smart systems come together in a symphony of harmony to improve safety, reliability, and excellence of operations. It demonstrates the capability of IoT in turning the system health philosophy on its head, and demonstrates that with out-of-the-box thinking and imagination, even student projects can be capable of bringing relief

from real-world engineering issues. The learning and experiences gained from this project will surely be a helpful stepping stone towards future implementation and development of smart maintenance technologies.

10. Future work

Expanding on the existing framework of the Automated Fault Detection System Utilizing IoT, future endeavors can investigate various enhancements and extensions to improve its functionality and adaptability. A significant area for development includes the incorporation of wireless sensor networks, facilitating uninterrupted communication among numerous devices without the need for physical connections. This enhancement would broaden the system's operational range, rendering it more appropriate for extensive industrial or infrastructure monitoring.

Future endeavors may also aim to integrate the system with cloud-based platforms, facilitating remote data access, secure storage, and enhanced data analytics. This integration will support long-term trend analysis and enable users to oversee multiple systems from various locations. Furthermore, the application of machine learning techniques could greatly enhance the system's capacity to identify intricate or early-stage faults by analyzing historical data patterns.

The creation of a mobile application represents a significant avenue, enabling users to receive real-time notifications, monitor system performance, and easily engage with the dashboard via their smartphones. Subsequent iterations of the system may also incorporate automated control features, allowing it to not only identify faults but also execute predetermined corrective measures, such as deactivating equipment or dispatching urgent alerts to maintenance personnel.

Ultimately, broadening the system's application to various sectors—such as smart homes, transportation networks, aerospace surveillance, and energy grids—will create new possibilities for inter-industry utilization. As IoT and sensor technology continue to progress, this initiative has the potential to transform into a more sophisticated and entirely autonomous fault detection and response mechanism.

11.Achievement and Award

Award Received: 1st Prize at Capstone Project Exhibition 2025

Event Date: 18/04/2025

Organized by: Lovely Professional University (LPU), School of Mechanical Engineering



12. References

- [1]. Kumar, Gajendra, et al. "Cost-Effective IoT-Based Home Automation Fault Detection and Monitoring System." *IUP Journal of Electrical & Electronics Engineering* 13.1 (2020).
- [2]. Ghadge, Gauri Sanjay, and Pragati N. Korde. "Iot based home automation with fault monitoring." *2022 10th International Conference on Emerging Trends in Engineering and Technology-Signal and Information Processing (ICETET-SIP-22)*. IEEE, 2022.
- [3]. Parashar, Shivam, et al. "Advanced IOT Based Home Automation." *International Journal for Advance Research and Development* 3.3 (2018): 113-116.
- [4]. Parashar, Shivam, Mohd Zaid, Niket Vohra, and Sujeet Kumar. "Advanced IOT Based Home Automation." *International Journal for Advance Research and Development* 3, no. 3 (2018): 113-116.